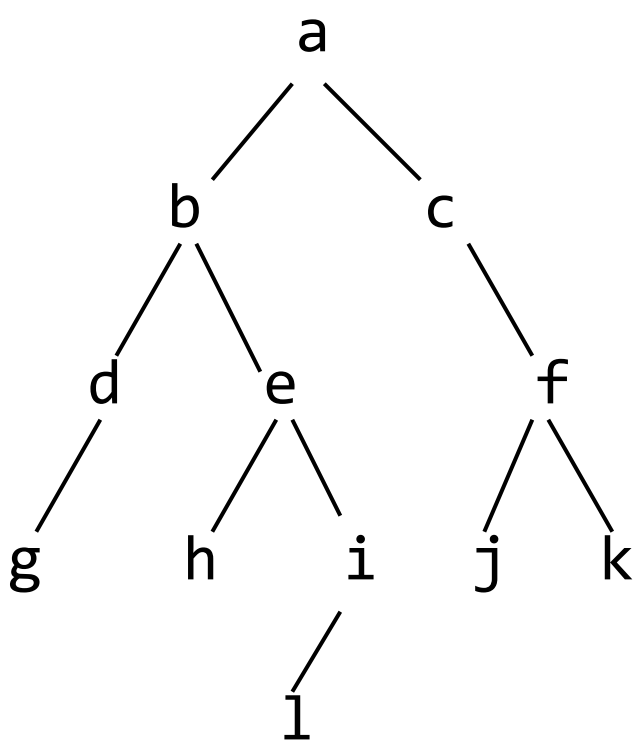
COMP 511

Computer Programming Foundations II

AVL (Adelson-Velsky and Landis) **tree**

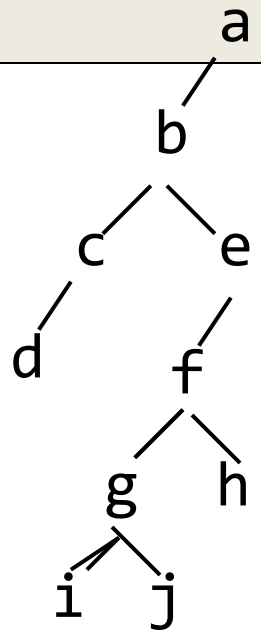
AVL tree



Balanced binary tree?

- The size of a binary tree is the number of nodes in it
  - tree (to the left) has size 12
- The depth of a node is its distance from the root
  - a is at depth zero
  - e is at depth 2
- The depth of a binary tree is the depth of its deepest node
  - This tree has depth 4

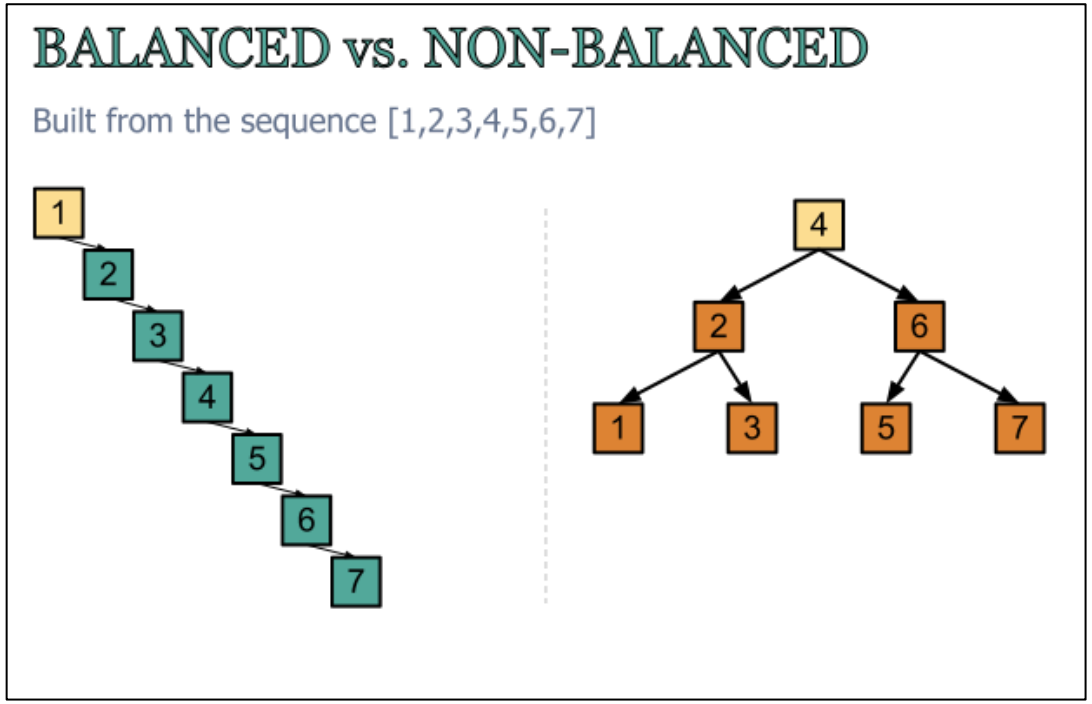
A binary tree is **balanced** if every level above the lowest is "full" (contains 2^n nodes, where n = node's depth)

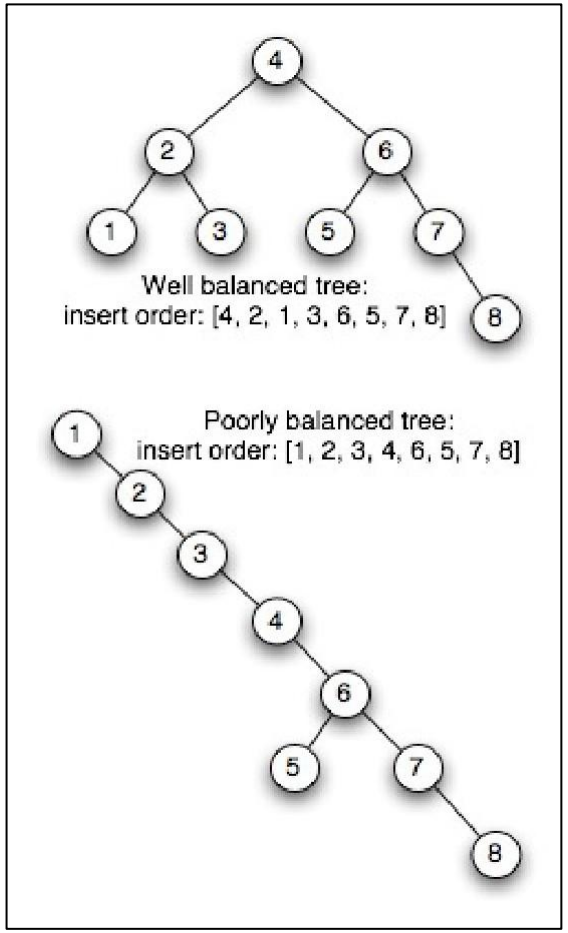


Unbalanced binary tree

AVL tree

The binary search tree is a decision tree, where the question asked at an internal node v is whether the search key k is less than, equal to, or greater than the key stored at v. Searching, insertion, and removal in a binary search tree is O(h), where h is the height of the tree. In the worst-case scenario (see image on left) search , insertion, and removal time is O(n), where the height of the tree is equal to n. Thus in some cases searching, insertion, and removal is no better than in a single-linked list. To prevent the worst case, we need to develop a rebalancing scheme to bound the height of the tree to **log n**. We can show that an **AVL (**Adelson-Velsky and Landis) **tree with n nodes has O(log(n)) height.**



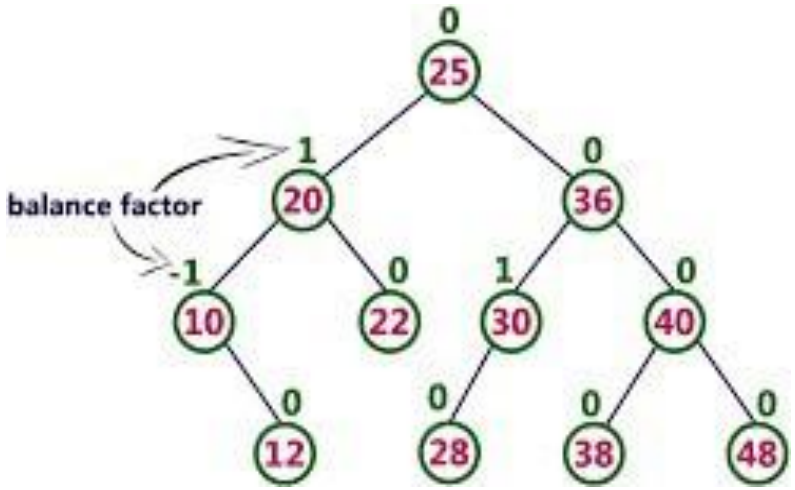


AVL tree

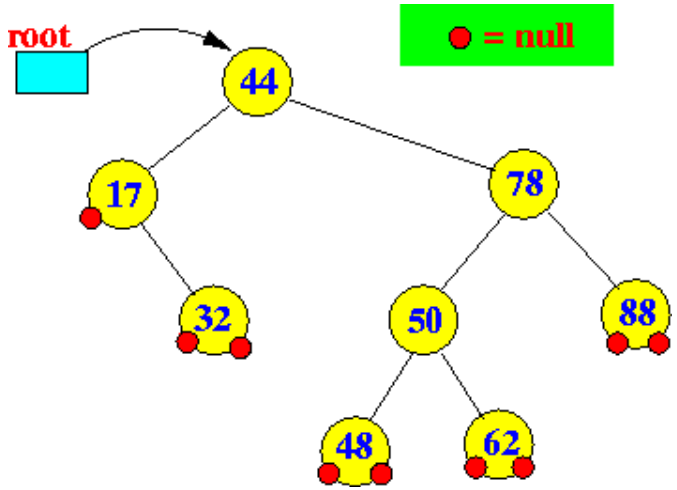
- The depth of a node is the number of edges from the root to the node.
- The height of a node is the number of edges from the node to the deepest leaf.
- The height of a tree is the height of the root.
- The root node has depth zero, leaf nodes have height zero, and a tree with only a single node (i.e. root) has depth and height zero. By definition, we consider an empty tree (null reference) to have height −1.
- A full binary tree is a binary tree in which each node has exactly zero or two children.
- A complete binary tree is a binary tree, which is completely filled, with the possible exception of the bottom level, which is filled from left to right.

An AVL Tree is a **self-balancing** binary search tree. This means that the heights of a given node's children trees are either the same or they differ by one. This is accomplished by rotating the tree nodes. Because the AVL Tree is self-balancing, all of its operations (insertion, lookup, and removal) operate in **O(log(n))** time.

BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))
or Height(RightSubtree(N)) – Height(LeftSubtree(N)). No matter, as long as the difference ∈ {-1,0,1}



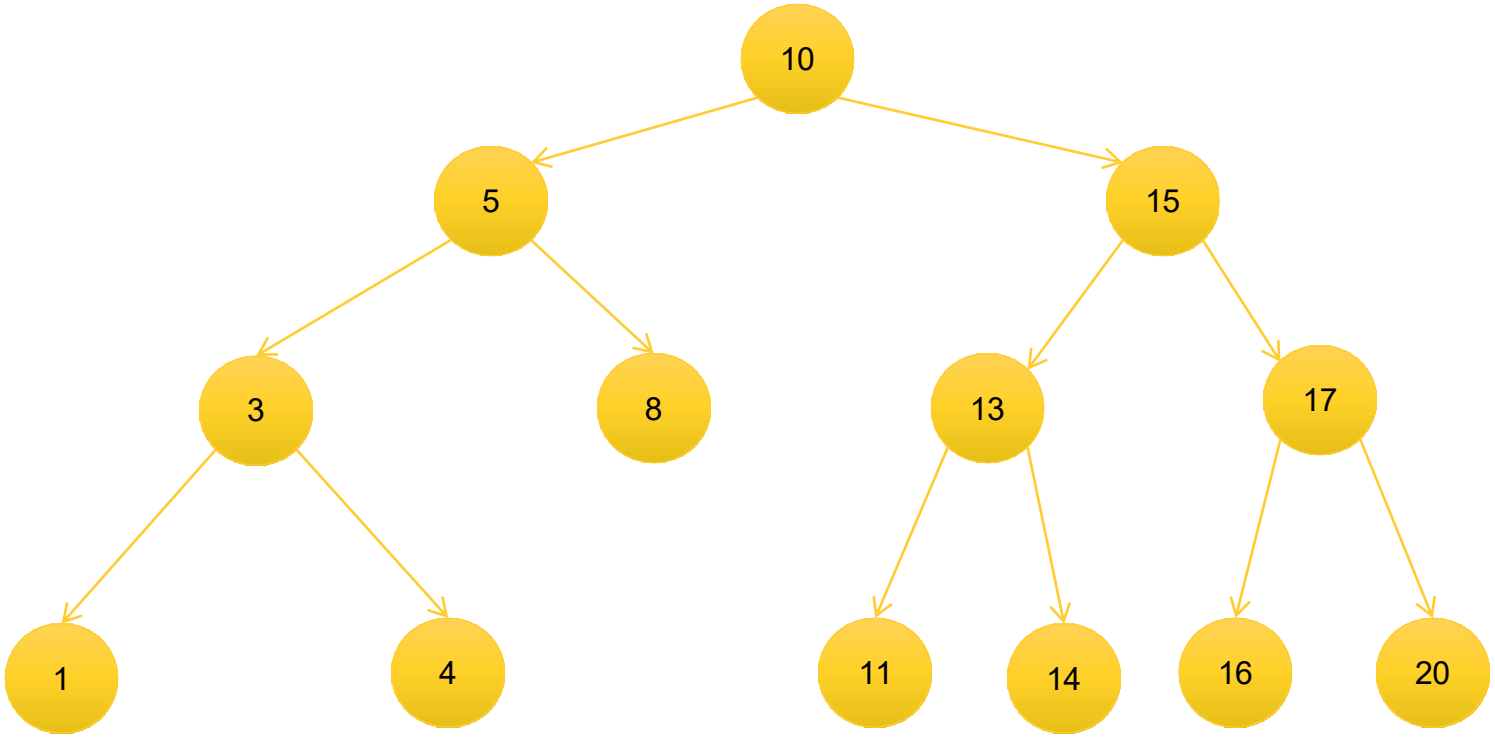
Provide the balance factor for each node:



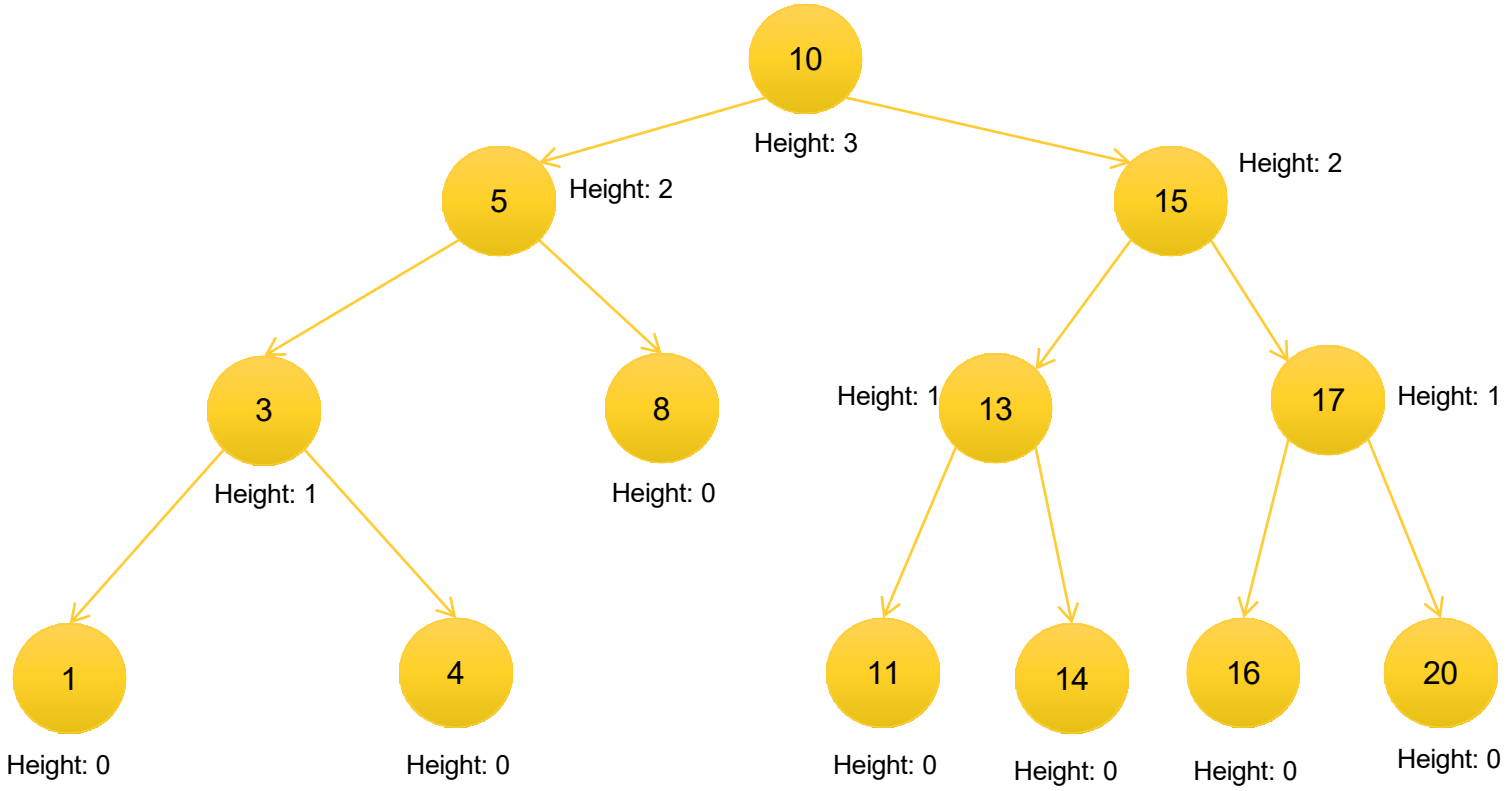
In this example,
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

AVL tree

Recall from earlier discussion that the height of a node is the number of edges from the node to the deepest leaf.
Provide the height of each node:



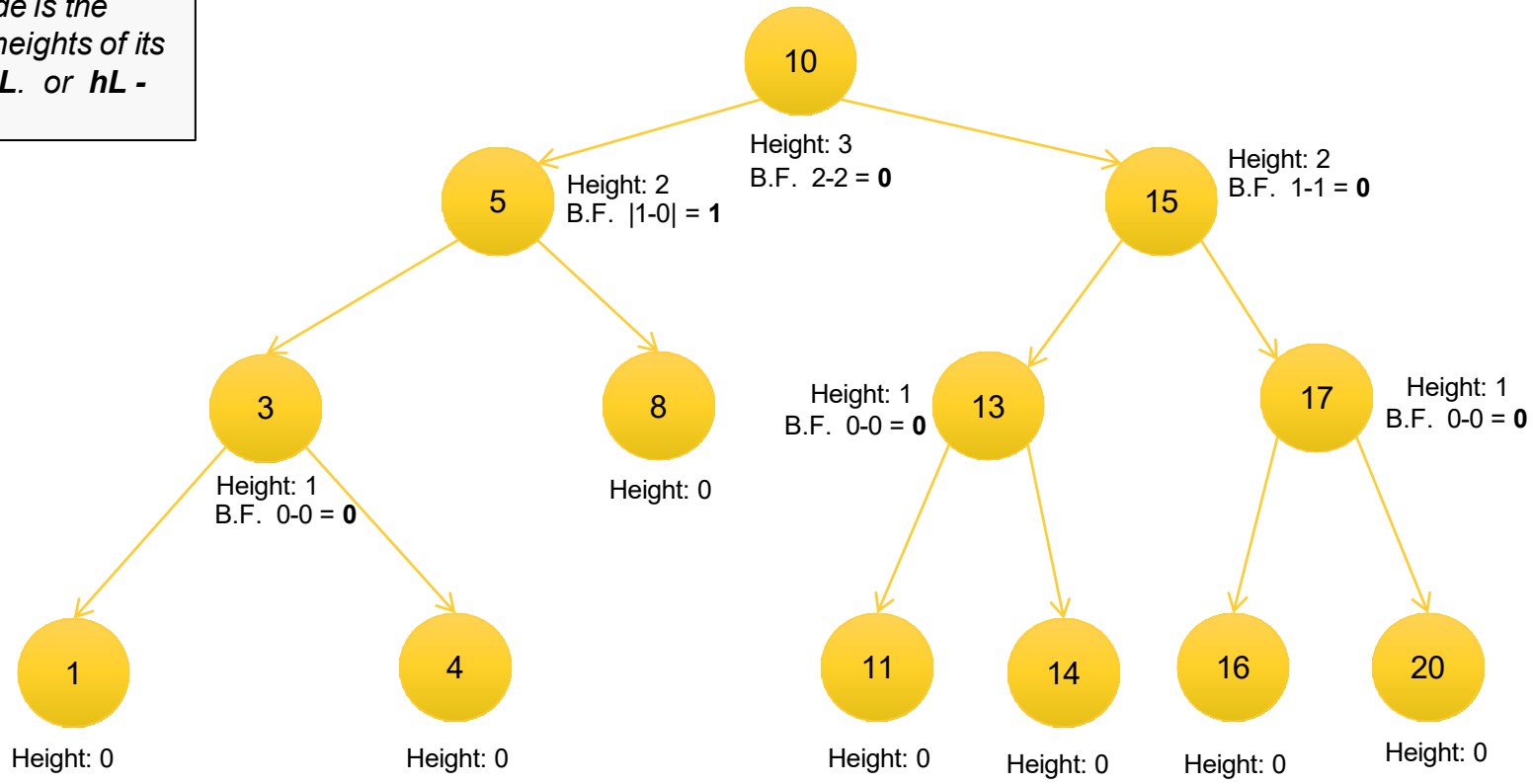
AVL tree



Leaf nodes have a height of 0.
By definition, we consider an empty tree (null reference) to have height −1.

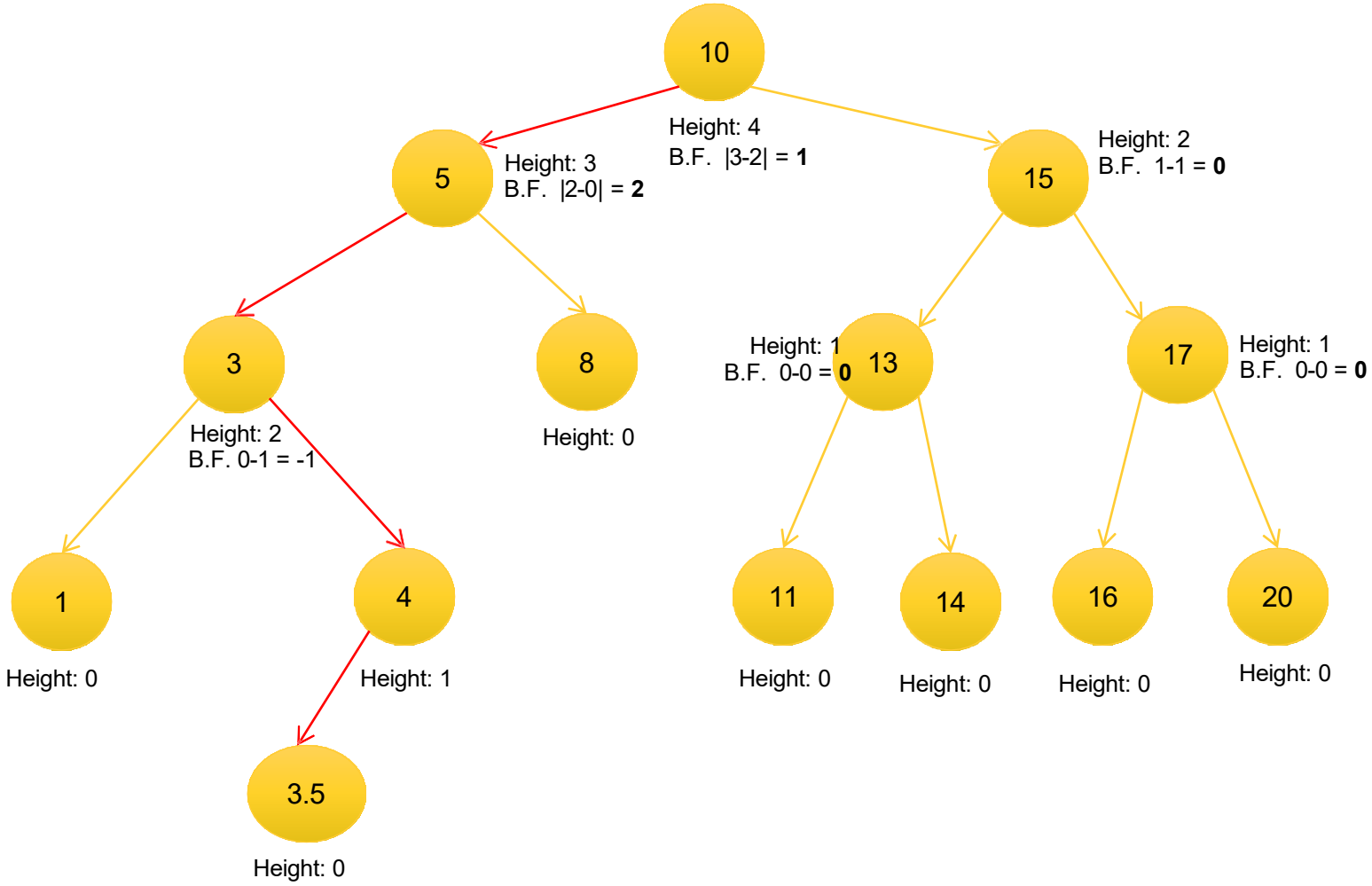
AVL tree

Let's compute the **balance factor** of each node. We will use the following formula:
Balance factor of a node is the absolute difference in heights of its two subtrees ***hR – hL****. or* ***hL - hR***



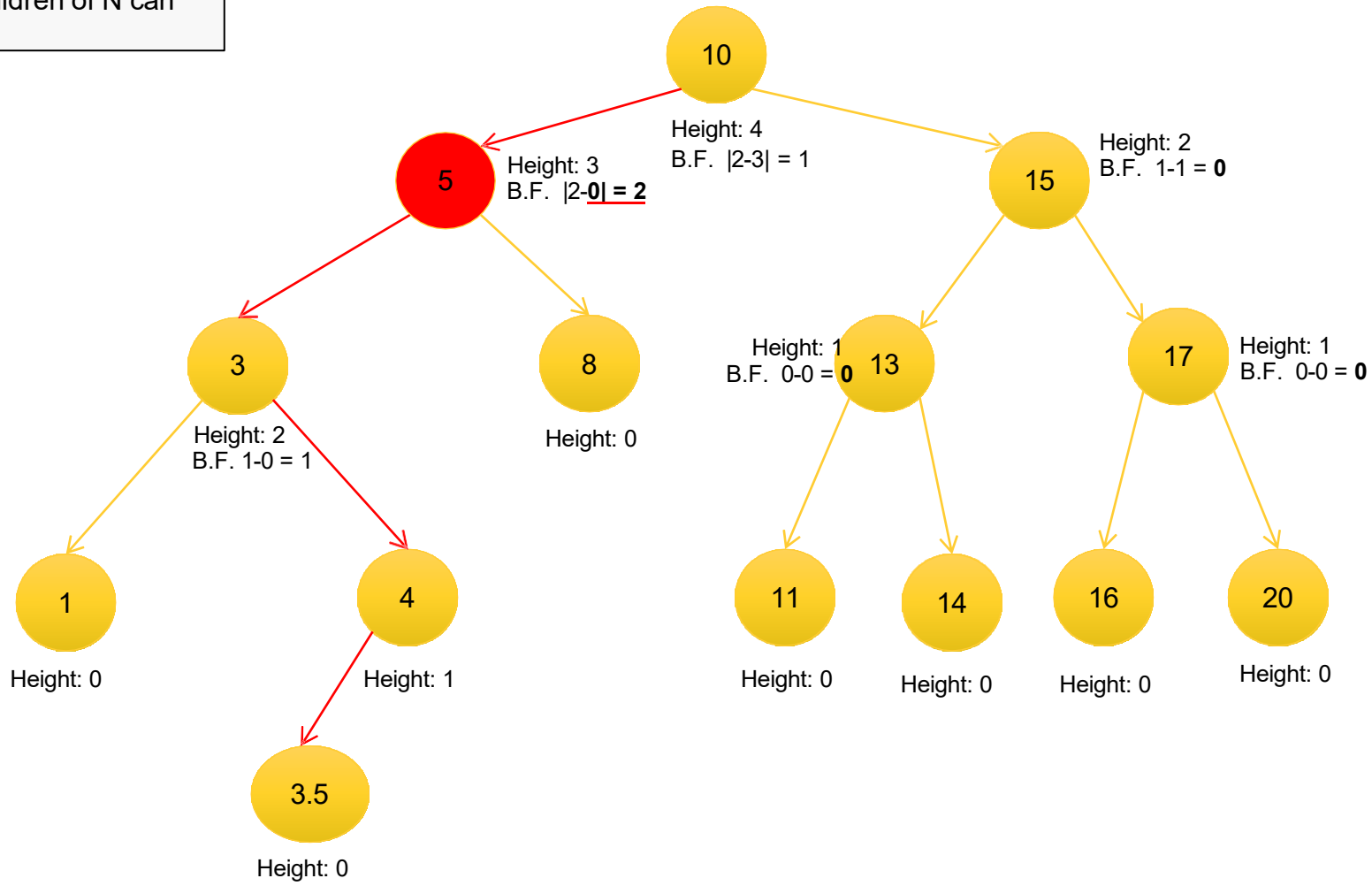
AVL tree

Let's say we insert(3.5)
We now need to update the height and balance factor of each node affected by the insertion.



AVL tree

Recall: An AVL Tree is a binary search tree **T** such that for every node **N** of T,
the heights of the children of N can differ by at most **1**



AVL tree

Insertion:

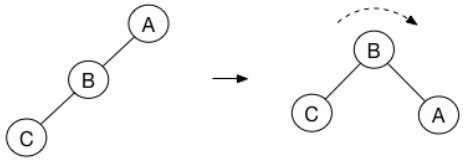
- After inserting a node, it is necessary to check each of the node's ancestors for consistency with the AVL rules.
- For each node checked, if the balance factor remains 1, 0, or -1 then no rotations are necessary. Otherwise, it's unbalanced.
- After each insertion, at most two tree rotations are needed to restore the entire tree.

To balance itself, an AVL tree may perform the following four kinds of rotations:

Rotations

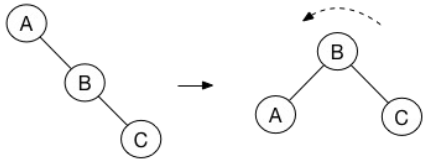
Single Rotation

Left Rotation (LL Rotation)



B becomes the new root.
A takes ownership of **B**'s right child as its left child (in this case, B's right child is null).
B takes ownership of **A** as its right child.

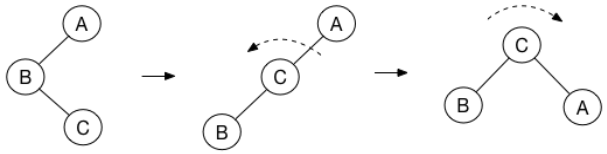
Right Rotation (RR Rotation)



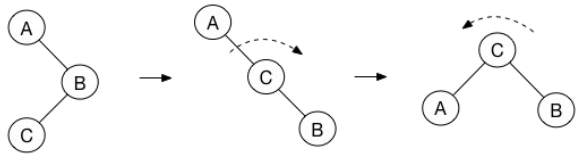
B becomes the new root.
A takes ownership of **B**'s left child, as its right child. (in this case, B's left child is null).
B takes ownership of **A**, as it's left child.

Double Rotation

Left Right Rotation (LR Rotation)



Right Left Rotation (RL Rotation)



AVL tree

Some terminology:

1. The **root node** is the first node above the inserted node that is unbalanced.
2. The **child node** is either the left or right child of the root node. The child node involved in rotation is the node on the path towards the recently inserted value.
3. In the case of single right and left rotations the pivot is the root node.
4. In the case of left-right and right-left rotations the pivot is the child node for the first rotation, and the original root node for the second rotation.

Rotating Right

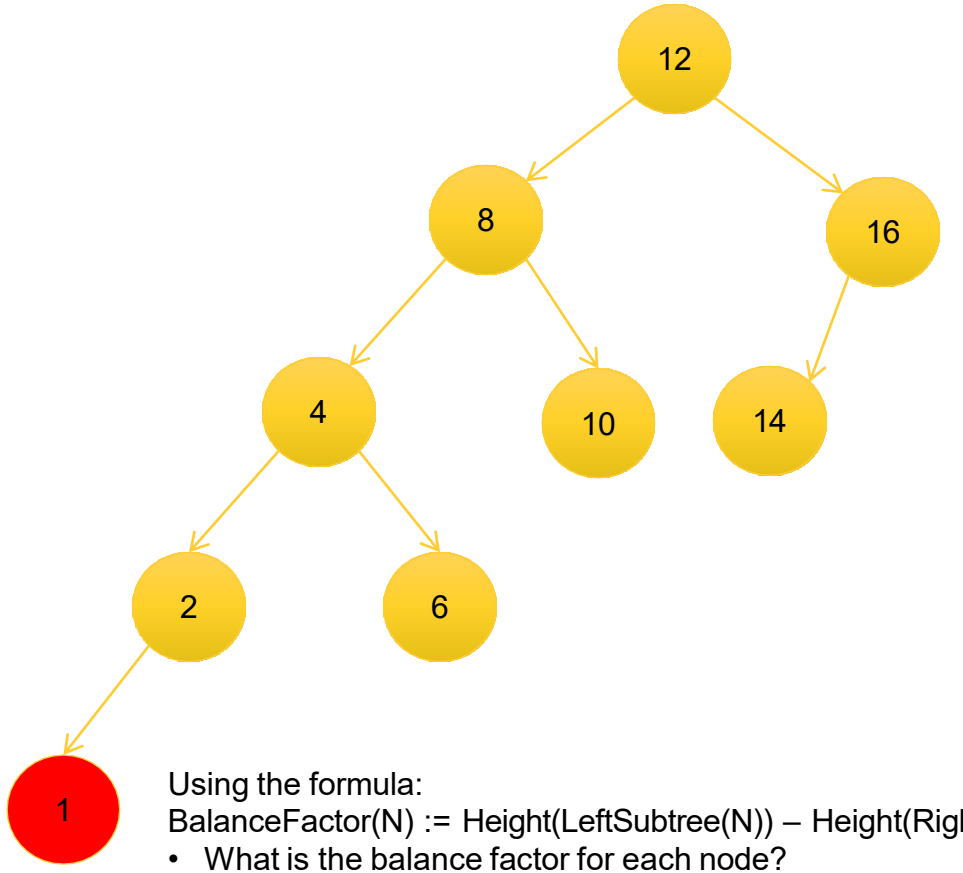
1. Save value of pivot.left (temp = pivot.left)
2. Set pivot.left to value of pivot.left.right
3. Set temp.right to pivot
4. Set pivot to temp

Rotating Left

1. Save value of pivot.right (temp = pivot. Right)
2. Set root. right to value of pivot.right.left
3. Set temp.left to pivot
4. Set pivot to temp

AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
```

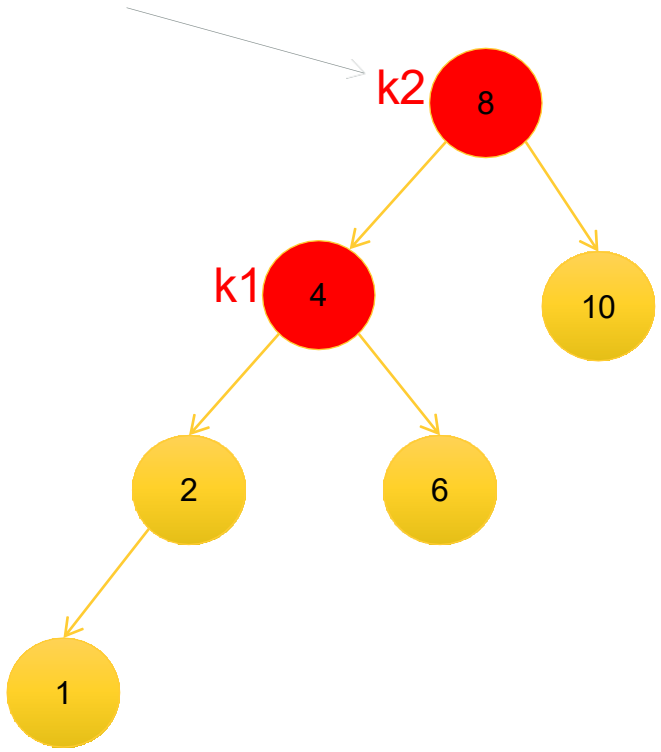


Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

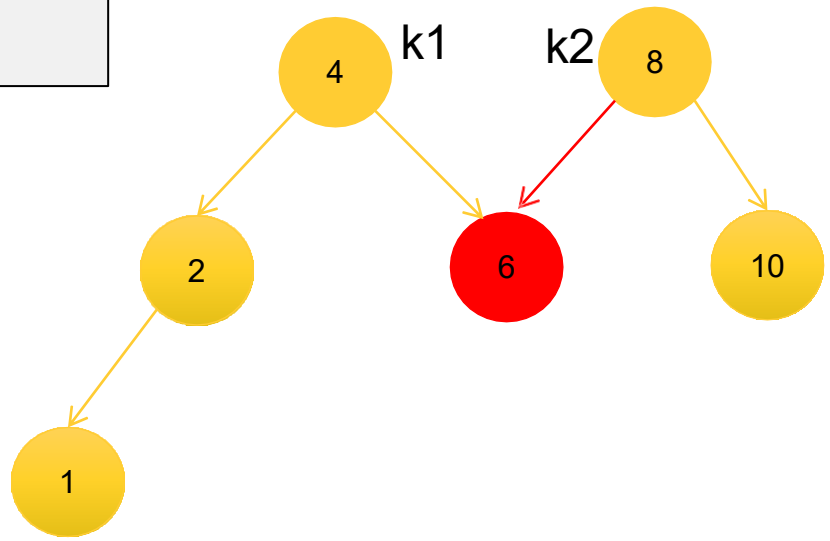
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
```



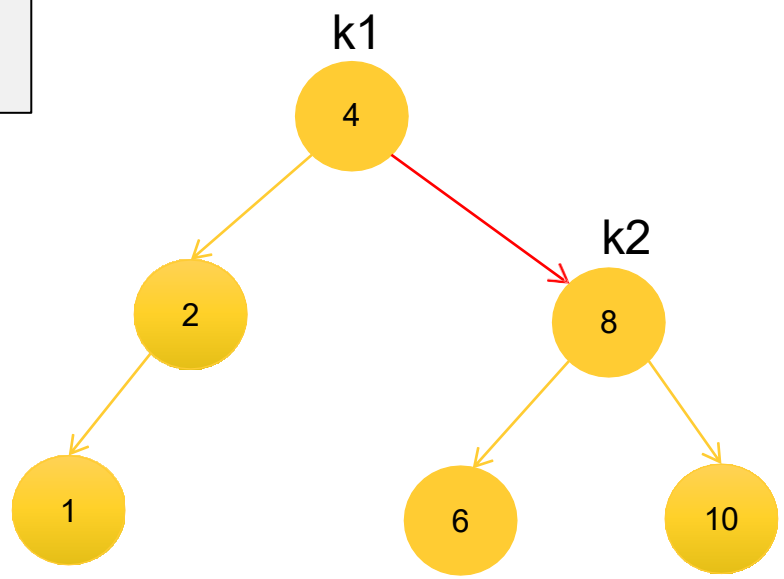
AVL tree

```
/**    (pseudocode)
  * Rotate binary tree node with left child.
  * For AVL trees, this is a single rotation for left/left.
  */
 static BinaryNode rotateWithLeftChild( BinaryNode k2 )
 {
     BinaryNode k1 = k2.left;
     k2.left = k1.right;
     k1.right = k2;
     return k1;
 }
```



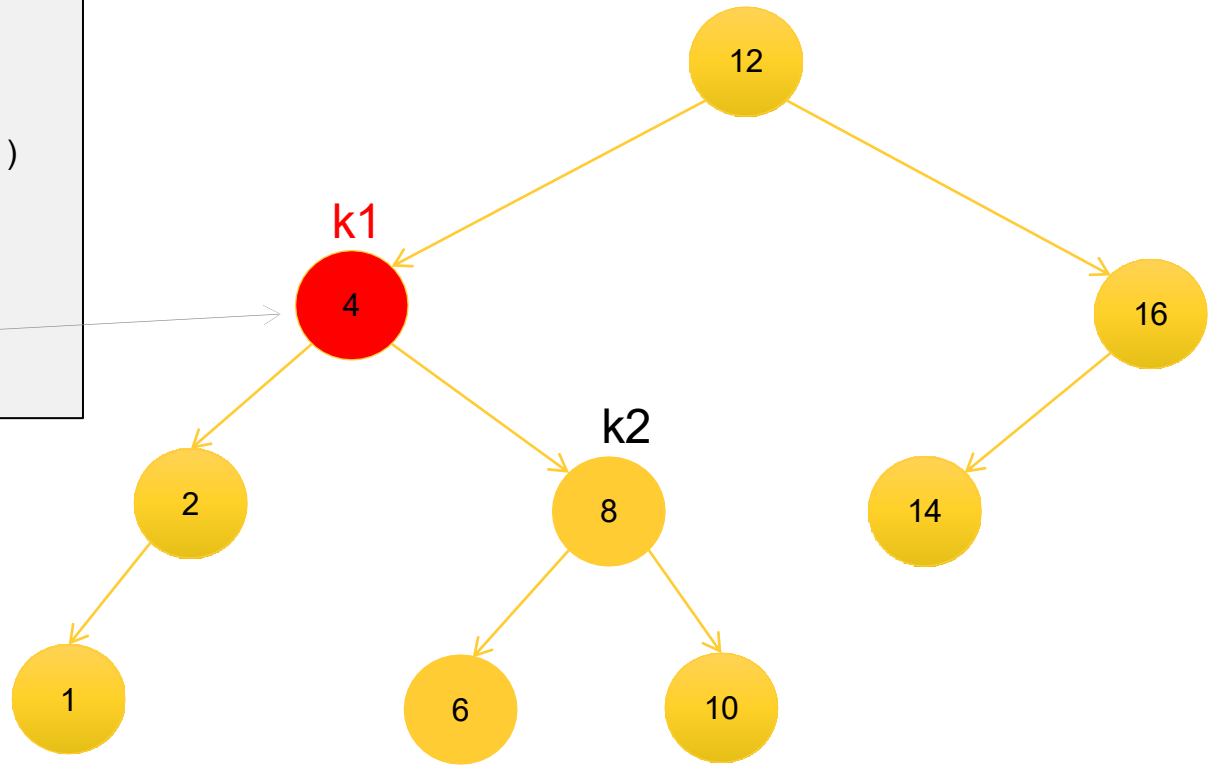
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
```



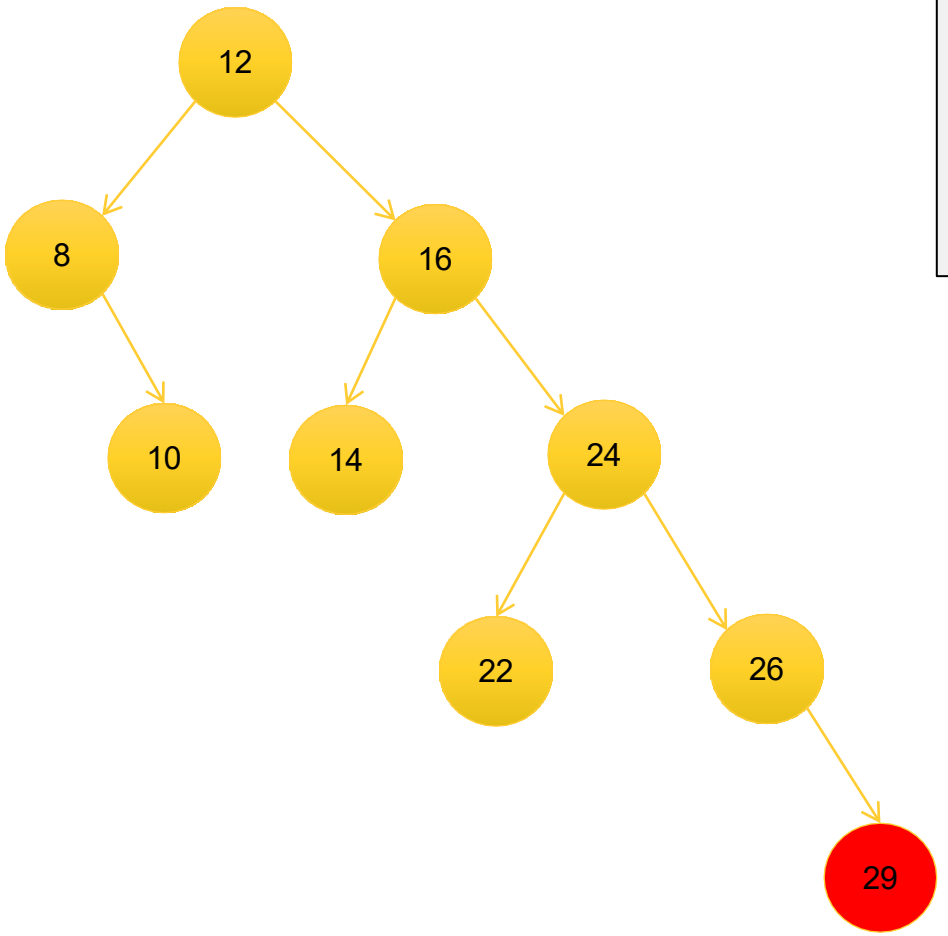
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
```



Left/Left Single Rotation
k1 becomes the new root.
k2 takes ownership of k1's right child (6) as its left child
k1 takes ownership of k2 as its right child.

AVL tree

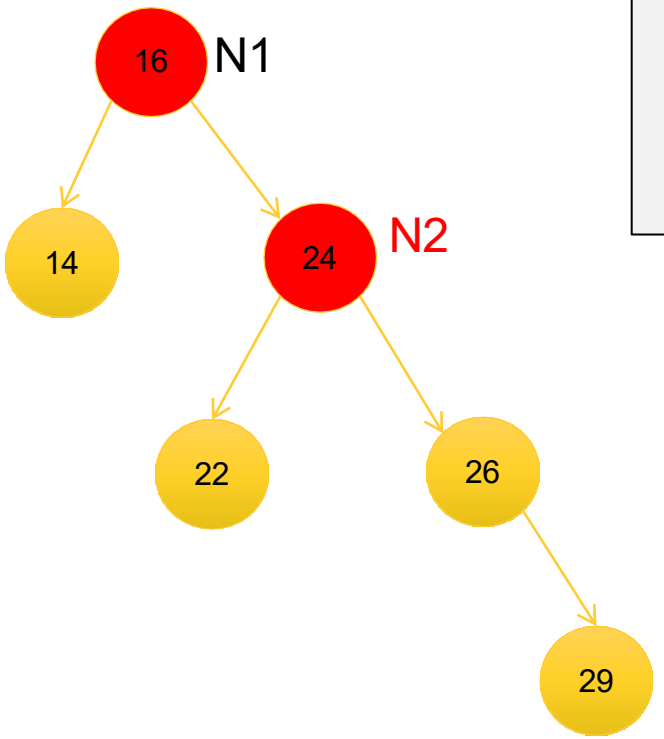



```
/**    (pseudocode)
 * Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

AVL tree



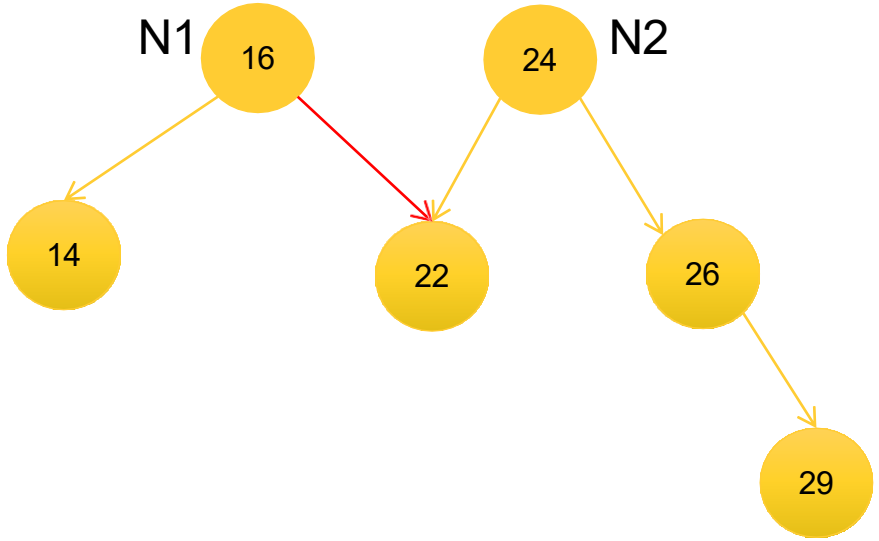

```
/**    (pseudocode)
 * Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
```

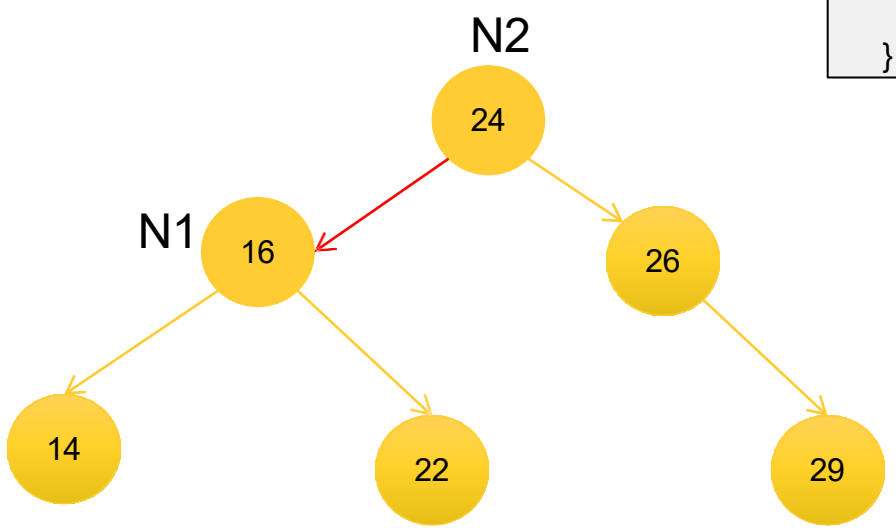


Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

AVL tree

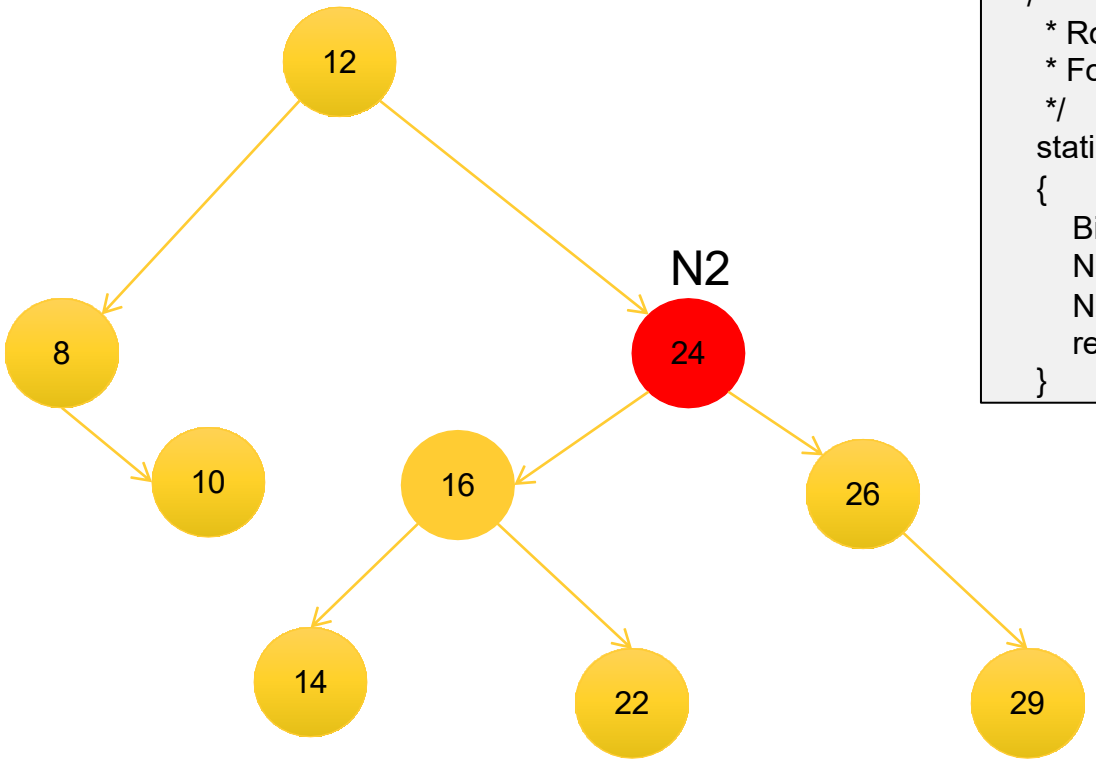
```
/**    (pseudocode)
 * Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
```



Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

AVL tree




```
/**    (pseudocode)
 * Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

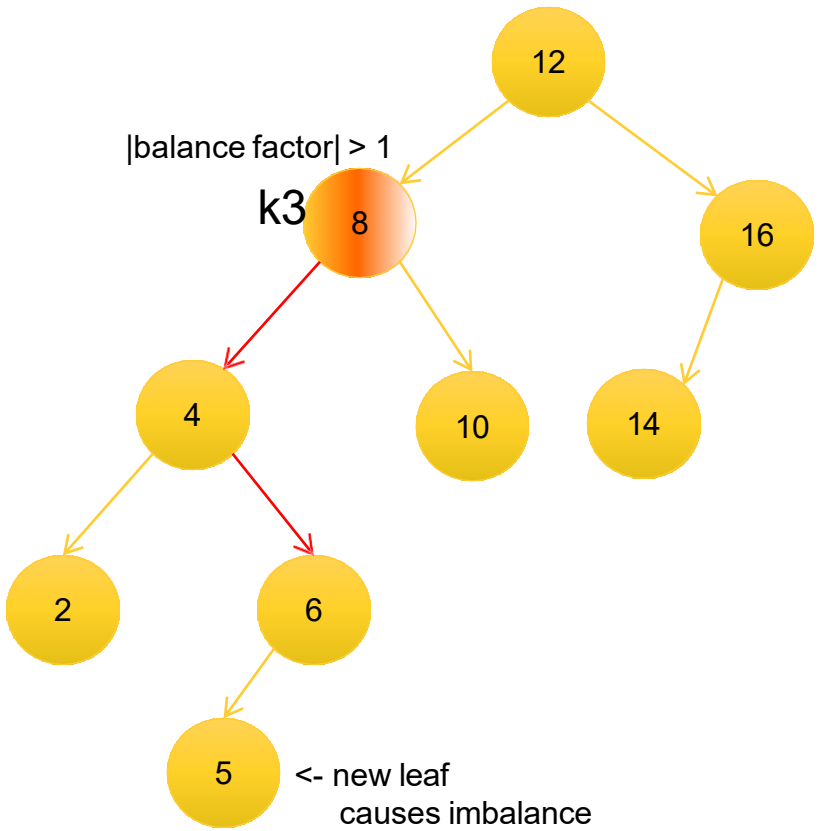
- What is the balance factor for each node?
- Which nodes are unbalanced?

AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

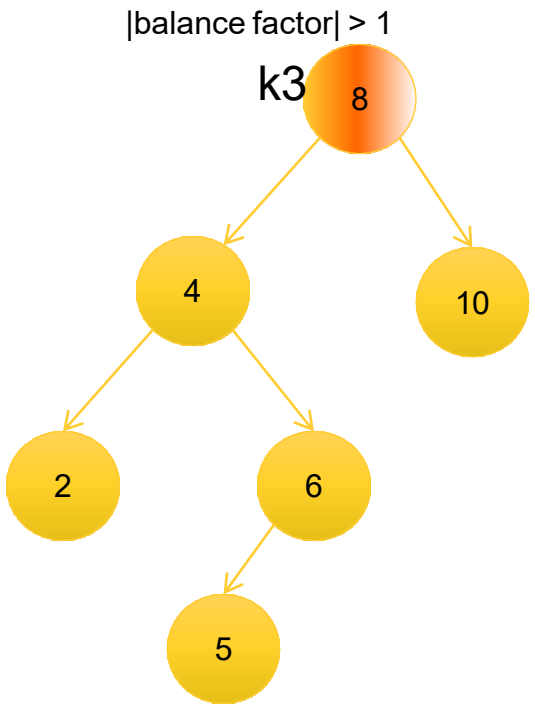


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```

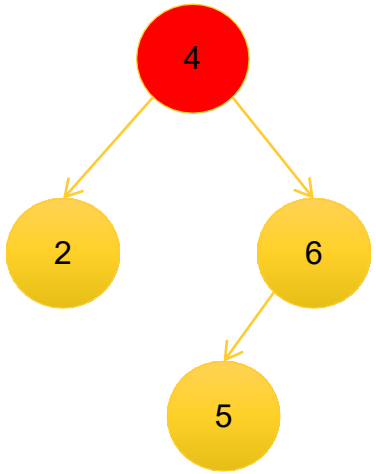
Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?



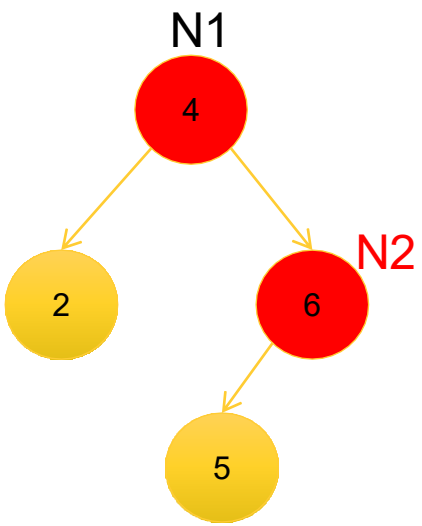
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



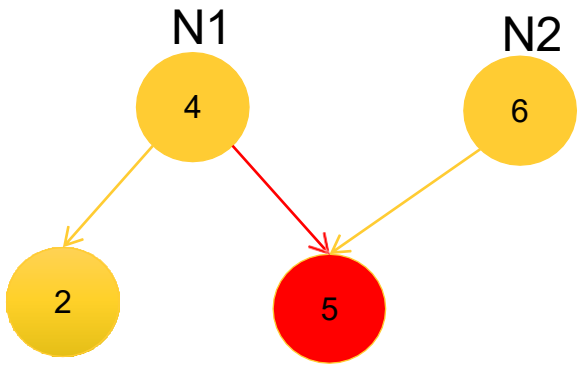
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



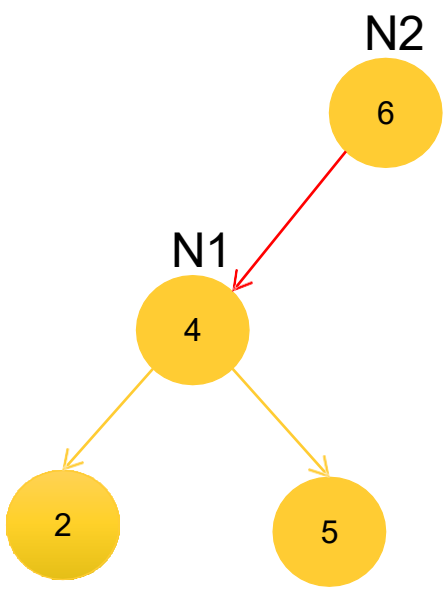
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



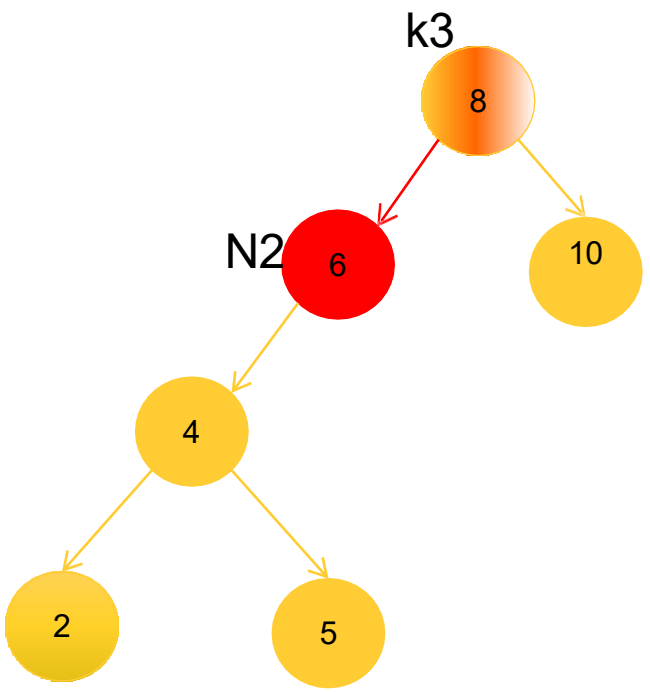
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



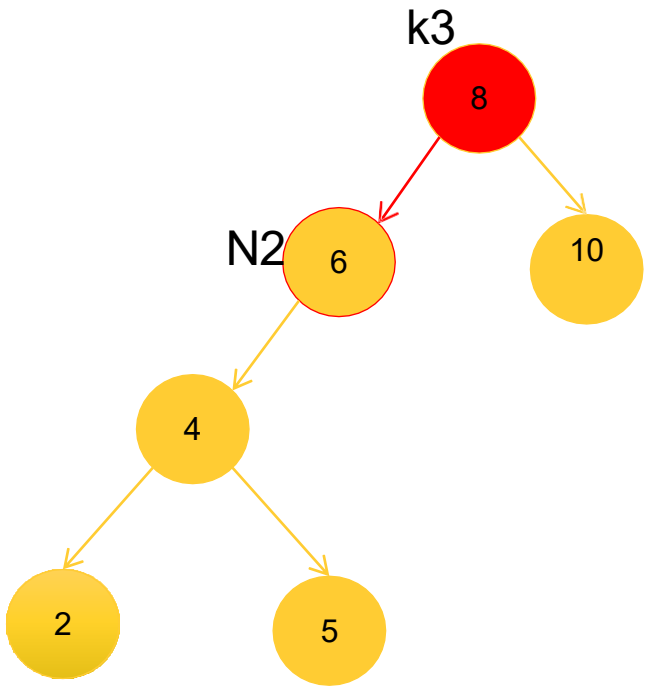
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3
)
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



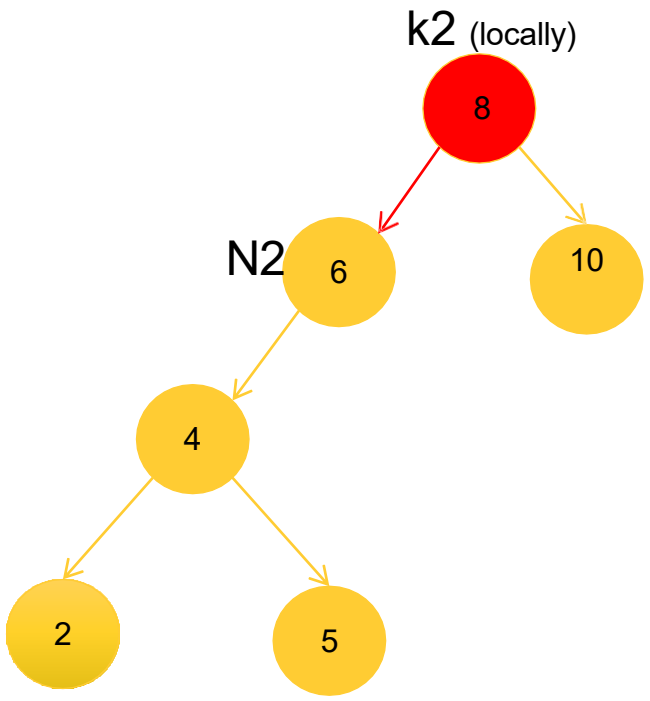
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



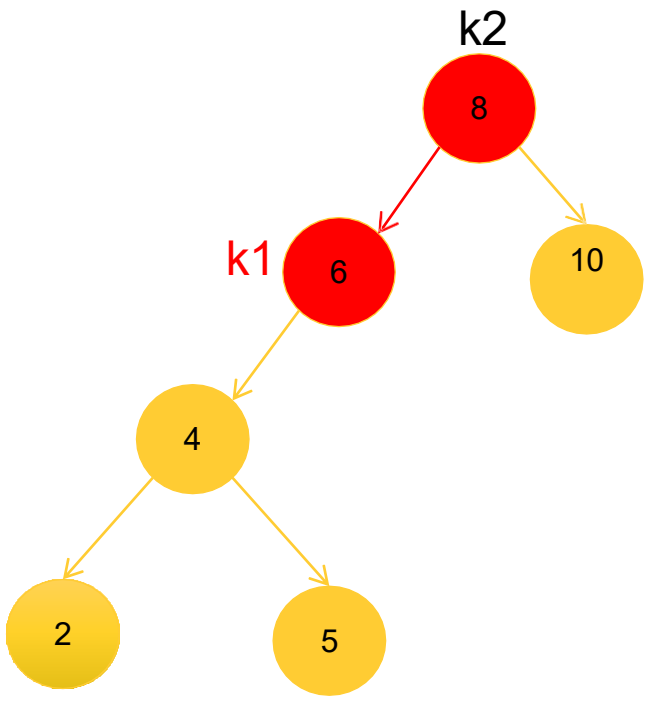
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



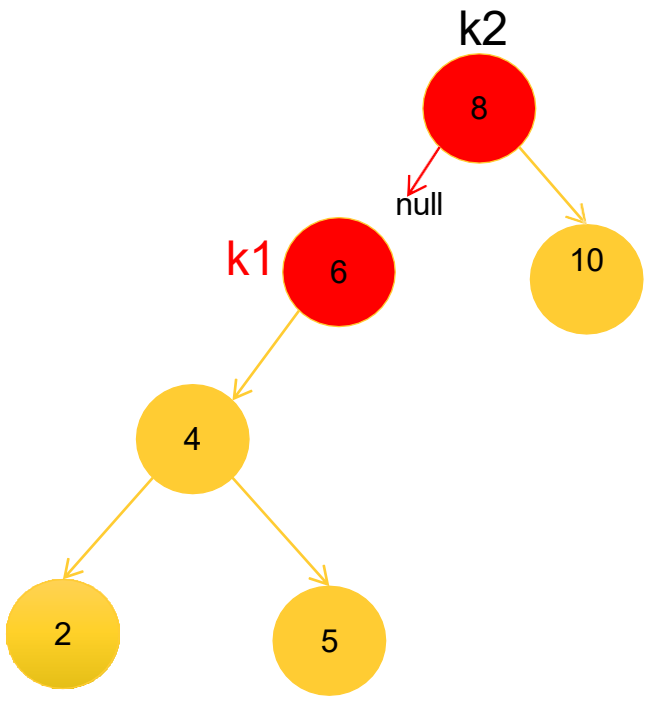
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



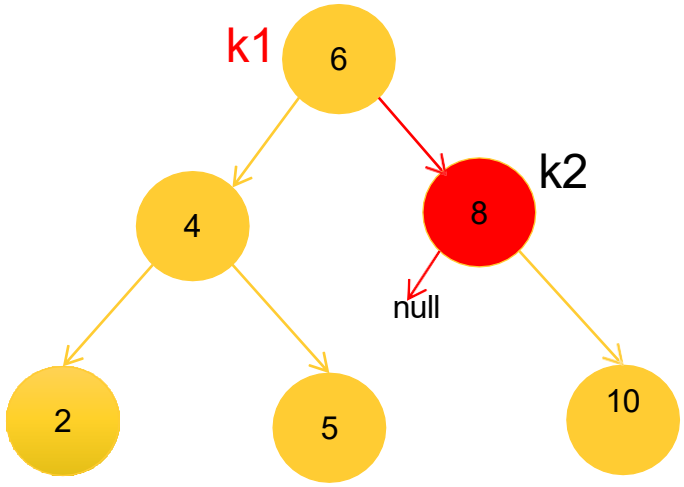
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



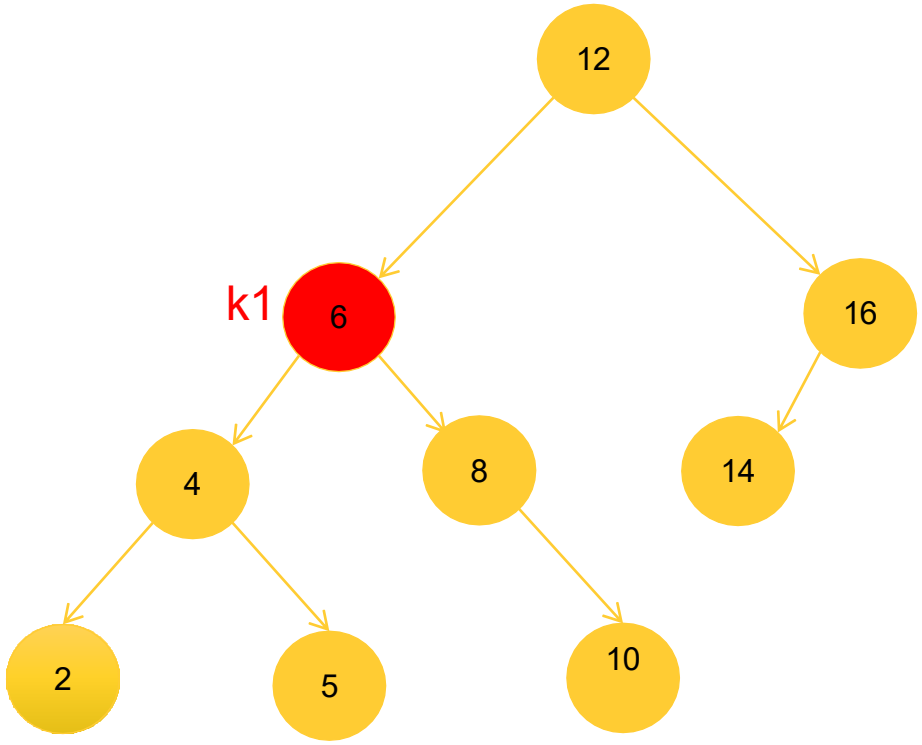
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



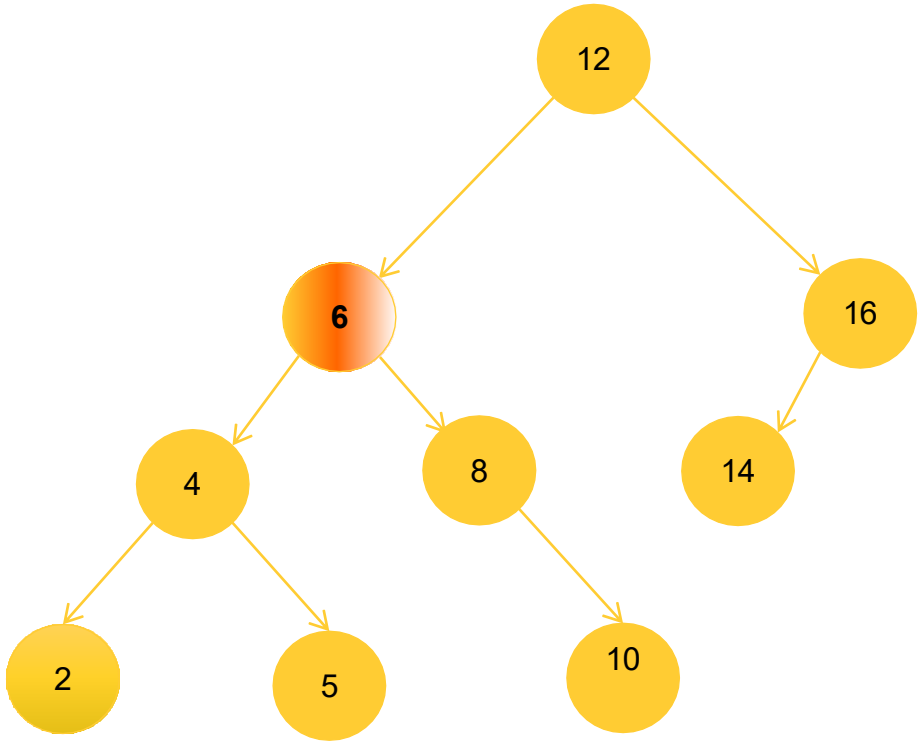
AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```



AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 */
static BinaryNode doubleRotateWithLeftChild( BinaryNode k3 )
{
    k3.left = rotateWithRightChild( k3.left );
    return rotateWithLeftChild( k3 );
}
```

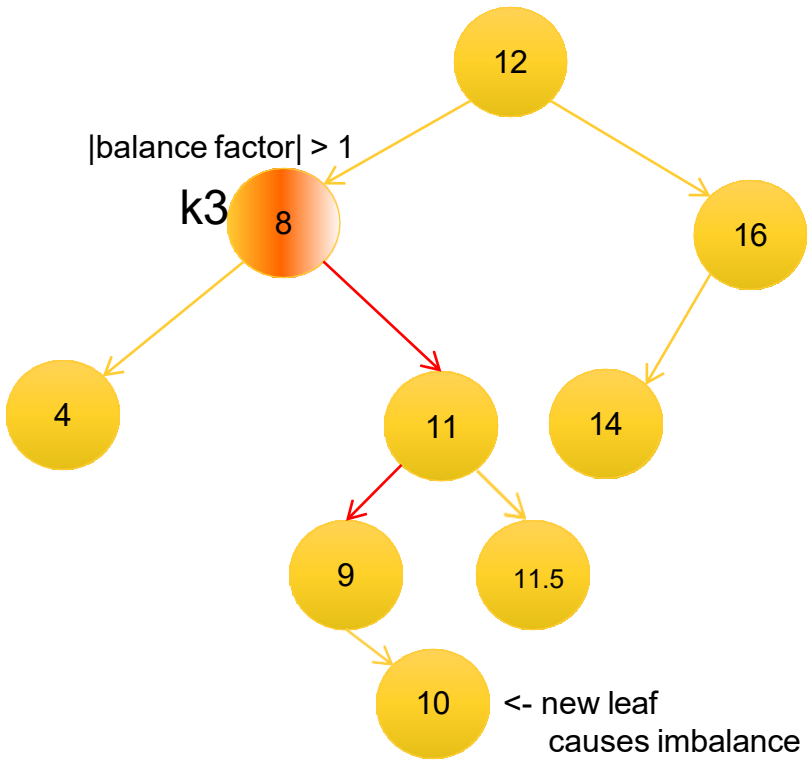


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

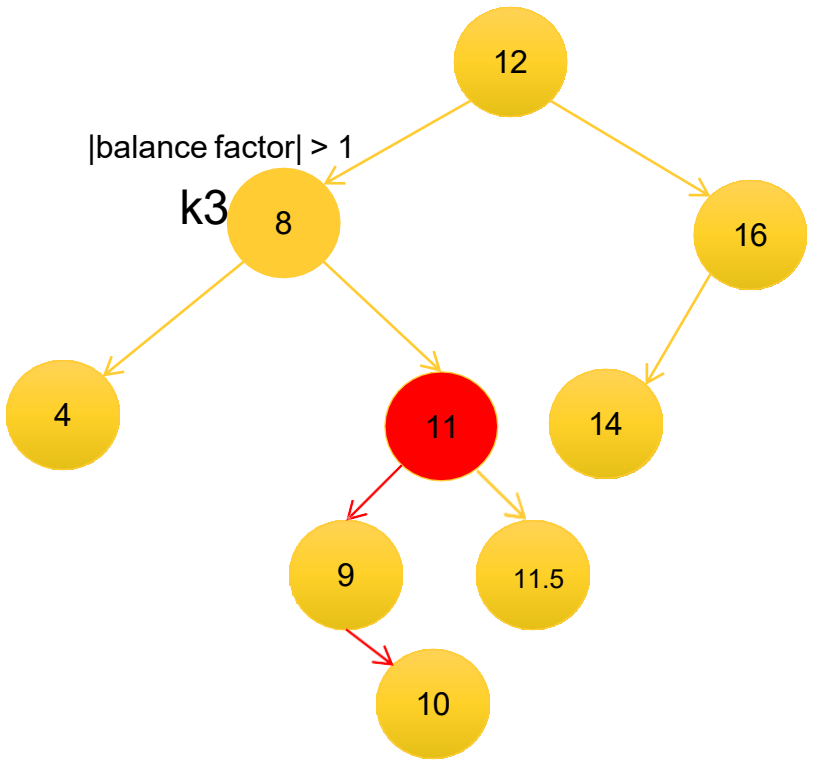


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

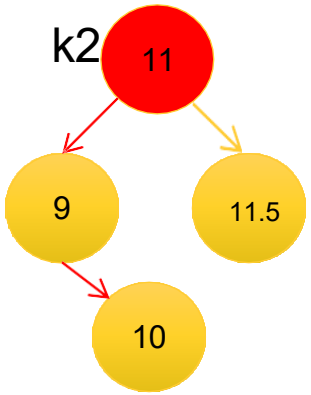


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

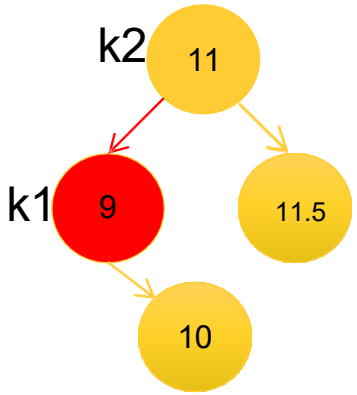


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

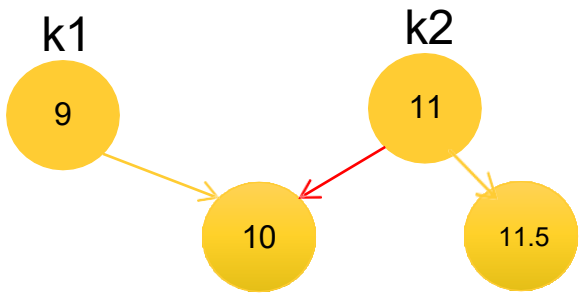


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

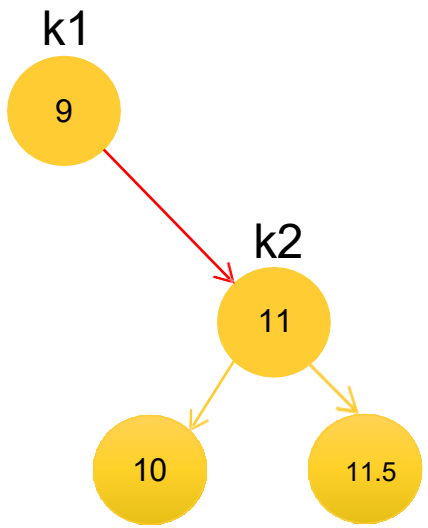


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

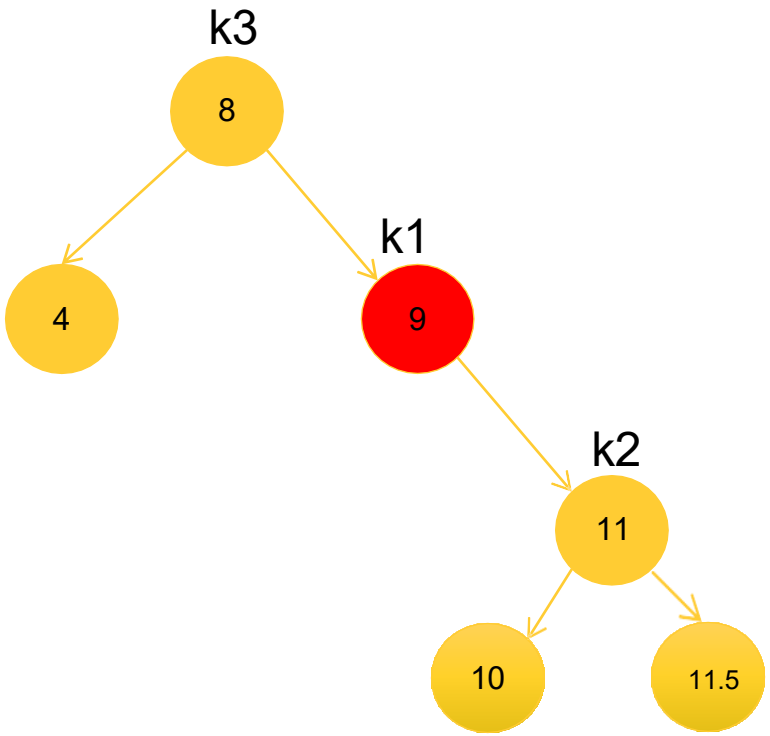


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

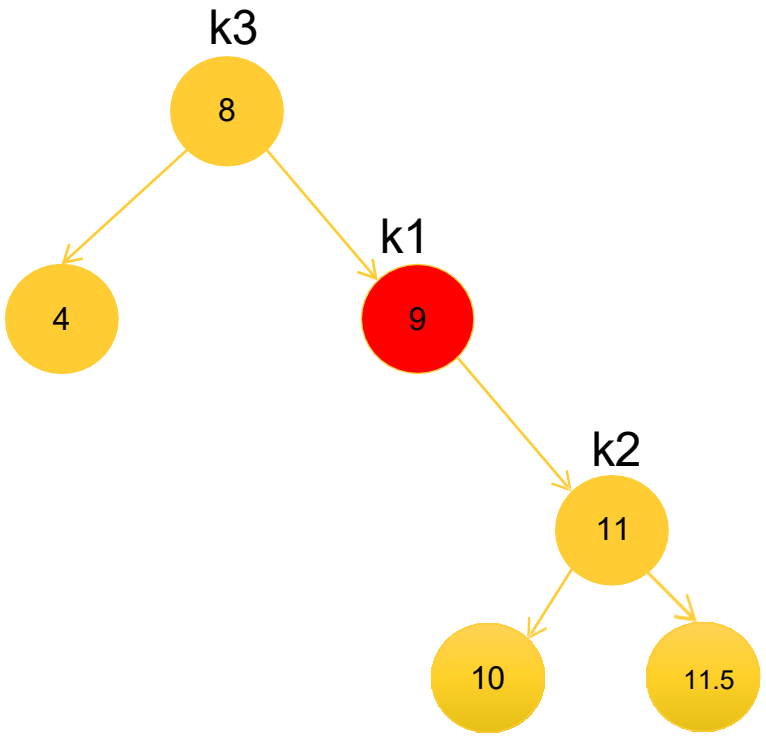


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

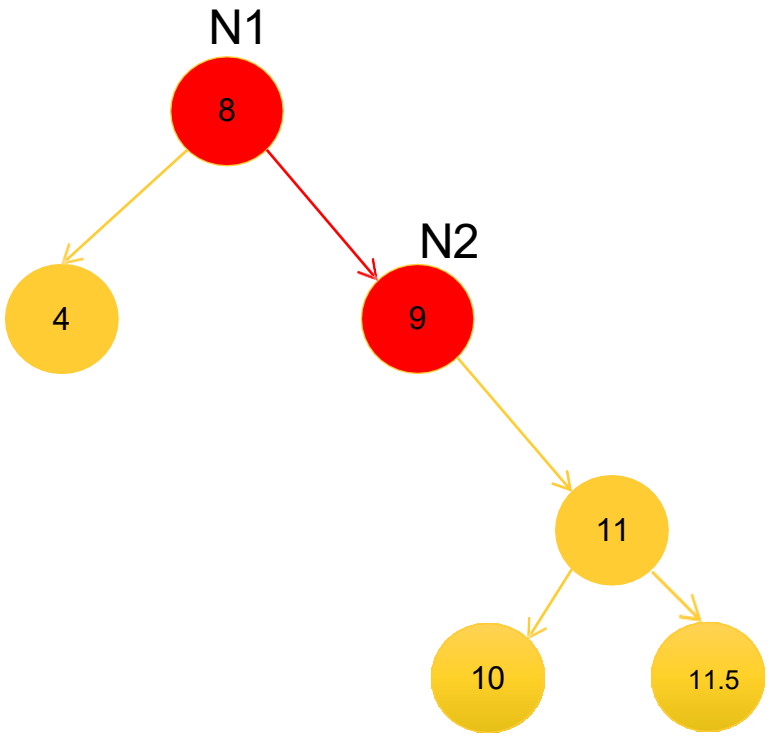


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

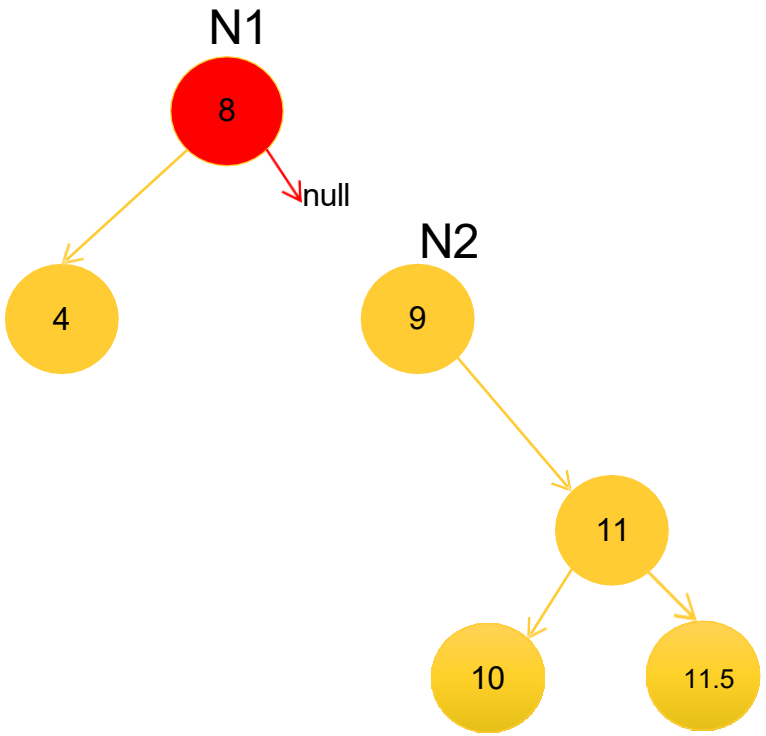


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?

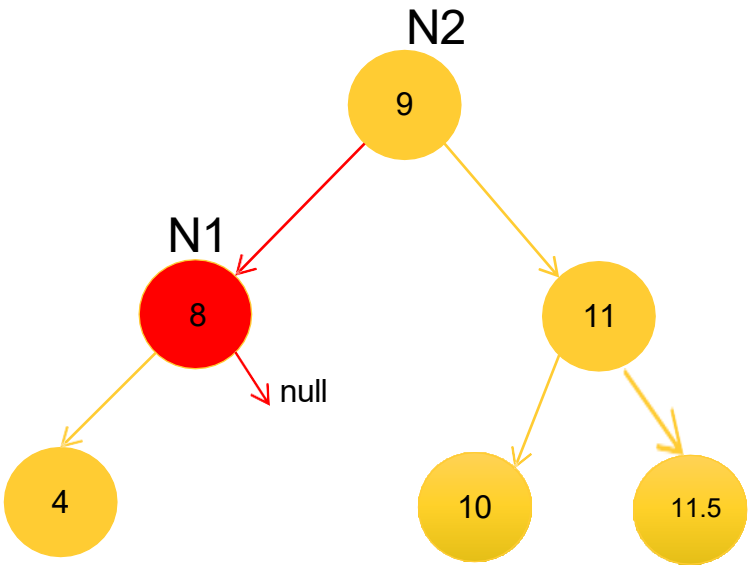


AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

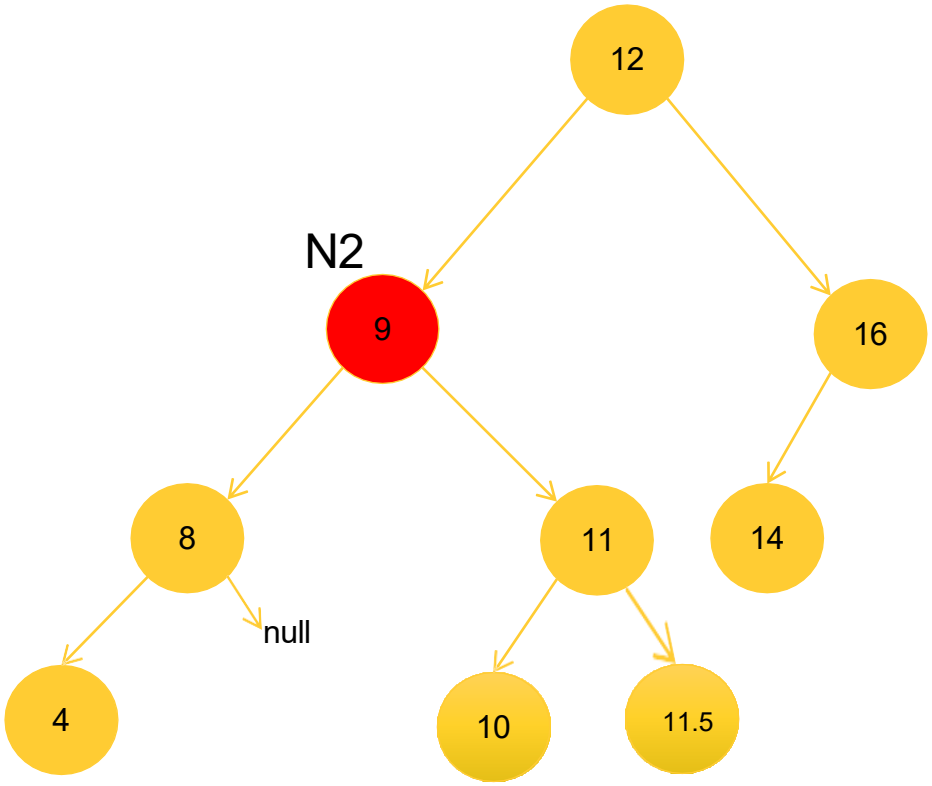
Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?



AVL tree

```
/**    (pseudocode)
 * Rotate binary tree node with left child.
 * For AVL trees, this is a single rotation for left/left.
 */
static BinaryNode rotateWithLeftChild( BinaryNode k2 )
{
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}
/** Rotate binary tree node with right child.
 * For AVL trees, this is a single rotation for right/right
 */
static BinaryNode rotateWithRightChild( BinaryNode N1 )
{
    BinaryNode N2 = N1.right;
    N1.right = N2.left;
    N2.left = N1;
    return N2;
}
/**
 * Double rotate binary tree node: first right child
 * with its left child; then node k1 with new right child.
 */
static BinaryNode doubleRotateWithRightChild( BinaryNode k3 )
{
    k3.right = rotateWithLeftChild( k3.right );
    return rotateWithRightChild( k3 );
}
```

Using the formula:
BalanceFactor(N) := Height(LeftSubtree(N)) – Height(RightSubtree(N))

- What is the balance factor for each node?
- Which nodes are unbalanced?